

Final Group Assignment: Statistical Audit of an Open-Source Project microsoft/vscode

KELOMPOK 12

Dosen Pengampu:

Muhammad Ridho Kurniawan Pratama, S.Kom., M.T.I



Intelligentia - Dignitas

Semester : 2
Mata Kuliah : Statiska dan Probabilitas

Anggota Kelompok:

- | | | |
|---------------------------|---|------------|
| 1. Arista Imanuela Berutu | – | 1519625011 |
| 2. Multazam Ahmad | – | 1519625033 |
| 3. Keymal Alghifary | – | 1519625044 |
| 4. Dhanu Refian Majid | – | 1519625059 |
| 5. Muhammad Fadhil | – | 1519625061 |

**SISTEM DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS NEGERI JAKARTA
2026**

1. Ringkasan Eksekutif

Audit statistik terhadap repositori VS Code dilakukan menggunakan data Pull Request, Issues, dan Commits yang diperoleh dari GitHub API. Analisis menunjukkan bahwa repositori memiliki tingkat keberhasilan merge yang tinggi, dengan estimasi probabilitas Pull Request berhasil di-merge sebesar 87,23% dan tingkat ketidakpastian yang rendah.

Hasil uji hipotesis menunjukkan adanya perbedaan signifikan antara kontribusi manusia dan bot. Merge rate kontributor manusia mencapai 89,86%, sedangkan bot hanya 70,45%, sehingga kontribusi manusia terbukti lebih sering berhasil diintegrasikan ke repositori.

Analisis issue melalui simulasi Monte Carlo menunjukkan bahwa seluruh issue dalam sampel dapat diselesaikan dalam waktu maksimal 12 hari, yang mengindikasikan proses penanganan issue berjalan responsif dan efisien. Selain itu, penerapan Bloom Filter dan optimasi MCMC menunjukkan potensi peningkatan pengelolaan repositori, terutama pada kualitas Pull Request yang dibuat oleh bot.

Secara keseluruhan, repositori VS Code menunjukkan kinerja yang baik dalam pengelolaan kontribusi dan penyelesaian issue, namun masih terdapat peluang perbaikan pada proses otomatisasi yang melibatkan bot.

2. Tinjauan Data

2.1 sumber data

Sumber: GitHub REST API v2022-11-28 (api.github.com)

Repositori yang diaudit: microsoft/vscode (Visual Studio Code)

Periode pengumpulan: 27 Mei 2026

Rentang tanggal data yang ada:

Dataset	Rentang / Ukuran
Pull Requests	1.300 baris
Issues	13 Mei 2026 – 28 Mei 2026 (590 issue)
Commits	1.300 baris

2.2 Variabel Utama yang Dianalisis

Variabel	Dataset	Tipe	Digunakan untuk
is_merged	Pull Requests	Boolean	RQ1 — MLE Bernoulli, CI
author_type	Pull Requests	Kategorikal	RQ2 — Z-test User vs Bot

days_to_close	Pull Requests	Numerik	EDA durasi PR
days_open	Issues	Numerik	RQ3 — Monte Carlo P(>12 hari)
created_at	Semua	Datetime	Analisis tren temporal
authored_at	Commits	Datetime	Pola aktivitas kontributor

2.3 Keterbatasan Data

Pertama, GitHub API membatasi pengambilan data sebesar 100 item per halaman dengan maksimum 1.300 item per endpoint, sehingga data yang diperoleh merupakan sampel dari keseluruhan history repository dan bukan data lengkap sejak repository dibuat. Kedua, rentang tanggal antar dataset tidak seragam — Issues hanya mencakup 15 hari dibandingkan Pull Requests dan Commits yang mencakup sekitar 23 hari, sehingga perbandingan langsung antar dataset harus dilakukan dengan hati-hati. Ketiga, kolom `merged_at` pada Pull Requests yang bernilai kosong tidak selalu berarti PR ditolak, melainkan bisa juga disebabkan oleh data yang belum terisi pada saat pengambilan. Keempat, data commits diambil dari branch `main` saja sehingga tidak merepresentasikan aktivitas di branch pengembangan lainnya. Kelima, GitHub API tidak membedakan secara eksplisit antara akun Bot dan User pada semua endpoint, sehingga identifikasi dilakukan berdasarkan kolom `author_type` yang bergantung pada label yang diberikan GitHub.

2.4 Eksplorasi Data (EDA)

Audit Struktur & Validasi Kelayakan Data

Sebelum analisis statistik dilakukan, dilakukan audit struktur terhadap ketiga dataset untuk memastikan kelayakan data sesuai kriteria yang ditetapkan. Hasil audit mengonfirmasi bahwa dataset Pull Requests memenuhi kriteria minimum 500 merged PR, dataset Issues memenuhi kriteria minimum 1.000 closed issues, dan seluruh dataset memiliki kolom timestamp yang lengkap mencakup `created_at`, `closed_at`, dan `merged_at` pada Pull Requests, `created_at`, `closed_at`, dan `updated_at` pada Issues, serta `authored_at` dan `committed_at` pada Commits. Pemeriksaan missing values pada kolom-kolom kritis juga dilakukan untuk memastikan tidak ada data yang hilang pada variabel yang akan digunakan dalam analisis statistik lanjutan.

Eksplorasi Pull Requests — Dasar RQ1

Analisis distribusi variabel `is_merged` dilakukan untuk memperoleh gambaran empiris proporsi Pull Request yang berhasil di-merge di repository microsoft/vscode. Dari total 1.300 Pull Request yang diambil, dihitung nilai estimasi awal probabilitas merge ($\hat{\theta}$) menggunakan formula MLE Bernoulli $\hat{\theta} = k/n$ di mana k adalah jumlah PR yang di-merge dan n adalah total PR. Nilai $\hat{\theta}$ ini selanjutnya diserahkan kepada Anggota B sebagai input utama untuk estimasi formal dan kepada Anggota C untuk konstruksi Confidence Interval. Selain itu, dilakukan analisis hubungan antara ukuran PR (`is_large`) dengan keberhasilan merge untuk melihat apakah kompleksitas perubahan kode berpengaruh terhadap kemungkinan PR diterima.

Eksplorasi Merge Rate User vs Bot — Dasar RQ2

Untuk menjawab RQ2, dilakukan pemeriksaan distribusi tipe kontributor (`author_type`) pada dataset Pull Requests. Merge rate dihitung secara terpisah untuk kelompok User dan Bot dengan menghitung nilai k (jumlah PR merged) dan n (total PR) masing-masing kelompok. Perbedaan merge rate antar kelompok divisualisasikan melalui bar chart perbandingan dan countplot status merge per tipe kontributor. Nilai k dan n untuk masing-masing kelompok selanjutnya diserahkan kepada Anggota D sebagai input Z-test dua sampel proporsi guna menguji apakah perbedaan yang terlihat secara visual tersebut signifikan secara statistik.

Analisis Distribusi Durasi Issue — Dasar RQ3

Untuk mendukung RQ3, dilakukan analisis mendalam terhadap distribusi variabel `days_open` yang merepresentasikan durasi sebuah issue tetap terbuka sebelum ditutup. Dihitung nilai skewness dan kurtosis distribusi sebagai dasar justifikasi pemilihan metode Simulasi Monte Carlo. Selain itu, dihitung estimasi empiris langsung $P(\text{issue} > 12 \text{ hari})$ dari data sebagai nilai baseline pembandingan terhadap hasil simulasi yang akan dilakukan Anggota E. Visualisasi histogram dengan garis threshold 12 hari beserta boxplot untuk deteksi outlier ekstrem turut disertakan. Parameter distribusi empiris mencakup nilai mean (μ), standar deviasi (σ), dan median `days_open` juga diekstrak dan diserahkan kepada Anggota E sebagai parameter input simulasi.

Eksplorasi Aktivitas Commits

Analisis pola aktivitas commit dilakukan untuk melengkapi gambaran umum perilaku kontributor di repositori microsoft/vscode. Distribusi commit berdasarkan jam dalam sehari (Hour of Day, UTC) dan hari dalam seminggu divisualisasikan untuk mengidentifikasi periode produktivitas tertinggi para kontributor. Temuan ini memberikan konteks tambahan mengenai dinamika pengembangan proyek secara keseluruhan.

3. Estimator parameter

3.1 Pertanyaan Penelitian

No. 1 Berapa estimasi peluang sebuah Pull Request di microsoft/vscode akan berhasil ter-merge?

3.2 Pemilihan Model

Distribusi Bernoulli pada Variabel `is_merged` merupakan variabel biner yang hanya memiliki dua kemungkinan nilai, yaitu 1 apabila suatu *pull request* (PR) berhasil ter-merge dan 0 apabila tidak. Karena tidak terdapat nilai di antara kedua kategori tersebut, distribusi yang paling sesuai untuk memodelkan satu kejadian biner adalah distribusi Bernoulli dengan satu parameter θ (theta) yang menyatakan peluang keberhasilan (ter-merge). Setiap PR dapat dianalogikan sebagai pelemparan koin tak seimbang yang menghasilkan keberhasilan dengan peluang θ . Estimasi nilai θ dilakukan menggunakan metode *Maximum Likelihood Estimation* (MLE), yang menghasilkan estimator $\hat{\theta} = k/n = 1.134/1.300 \approx 0,8723$.

Distribusi Beta sebagai Posterior Bayesian Untuk melengkapi pendekatan estimasi titik dengan kerangka Bayesian, digunakan distribusi Beta sebagai *posterior*. Analisis dimulai dengan *prior* Beta(1,1), yang berarti sebelum data diamati seluruh nilai θ dari 0 hingga 1 dianggap memiliki kemungkinan yang sama (bersifat uniform tanpa asumsi awal). Setelah data diamati, *prior*

tersebut diperbarui menjadi $\text{Beta}(\alpha, \beta)$ dengan $\alpha = k + 1 = 1.135$ dan $\beta = m + 1 = 167$. Distribusi Beta ini menggambarkan tingkat keyakinan terhadap nilai θ , bukan sekadar satu angka estimasi tunggal. Oleh karena jumlah data yang besar (1.300 observasi), distribusi *posterior* yang dihasilkan menjadi sangat tajam di sekitar nilai 0,87, yang menunjukkan tingkat keyakinan yang tinggi. Pemilihan distribusi Beta didasarkan pada sifatnya sebagai *prior* konjugat bagi distribusi Bernoulli/Binomial. Secara matematis, apabila *prior* berupa distribusi Beta dan data mengikuti distribusi Bernoulli, maka *posterior* yang dihasilkan dipastikan juga berupa distribusi Beta.

3.3 Langkah Penurunan MLE

Penurunan MLE Bernoulli

1. **Likelihood:** $L(\theta) = \theta^k \cdot (1-\theta)^{(n-k)}$
2. **Log:** $\ell(\theta) = k \cdot \ln(\theta) + (n-k) \cdot \ln(1-\theta)$
3. **Turunkan = 0:** $k/\theta - (n-k)/(1-\theta) = 0$
4. **Selesaikan:** $k(1-\theta) = (n-k)\theta \rightarrow k = n\theta \rightarrow \theta = k/n$

Penurunan MLE Poisson

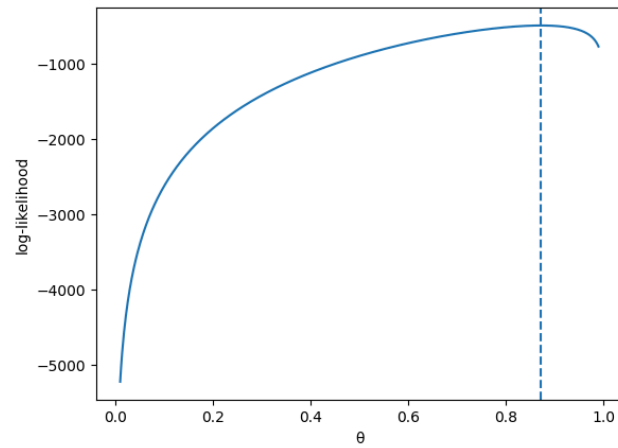
1. **Likelihood:** $L(\lambda) = (\lambda^{\sum x_i} \cdot e^{-(n\lambda)}) / \prod x_i!$
2. **Log:** $\ell(\lambda) = (\sum x_i) \cdot \ln(\lambda) - n\lambda - \ln(\prod x_i!) \rightarrow$ suku terakhir konstanta (tanpa λ)
3. **Turunkan = 0:** $(\sum x_i)/\lambda - n = 0$ (konstanta tadi hilang karena turunannya nol)
4. **Selesaikan:** $\hat{\lambda} = \sum x_i / n = \bar{x}$ (rata-rata sampel)

3.4 Hasil Estimasi

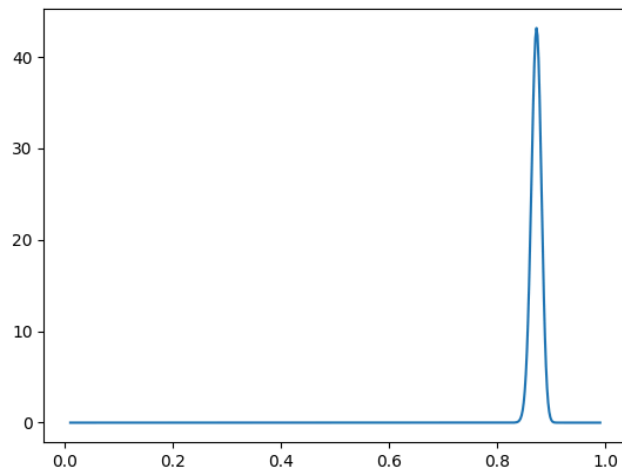
Data input: $n = 1.300$ PR, $k = 1.134$ PR ter-merge, $m = 166$ PR tidak ter-merge.

Besaran	Nilai
θ (MLE Bernoulli)	$k / n = 1.134 / 1.300 \approx 0,8723$
α (Beta posterior)	$k + 1 = 1.135$
β (Beta posterior)	$m + 1 = 167$
Mode Beta posterior	$(\alpha - 1) / (\alpha + \beta - 2) \approx 0,8723$
Mean Beta posterior	$\alpha / (\alpha + \beta) \approx 0,8717$

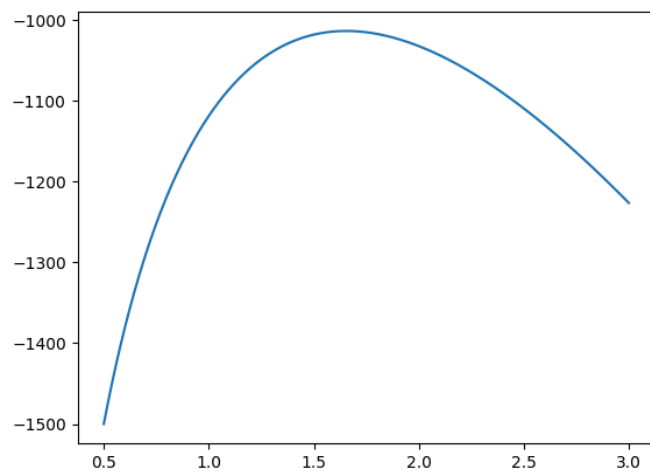
3.5 Visualisasi



Kurva log-likelihood- gamabar 3.5.1



Distribusi Beta posterior -gamabar 3.5.2



jumlah komentar per issue dengan poisson- gambar 3.5.3

3.6 Interpretasi

Pada puncak kurva log-likelihood terdapat nilai $\hat{\theta}$, yaitu sekitar 0,87. Nilai ini merupakan estimasi peluang yang dihitung dari data 1.300 PR. Angka 0,87 berarti kira-kira 87 dari setiap 100 PR di vscode akhirnya berhasil ter-merge. Lonjakan distribusi Beta posterior yang sempit disebabkan oleh banyaknya jumlah data. Semakin banyak data yang diamati, semakin yakin kita terhadap nilai peluangnya, sehingga distribusinya menjadi sempit dan tinggi. Sebaliknya, seandainya datanya hanya 20 PR, lonjakan Beta akan menjadi lebar dan pendek — artinya tingkat ketidakpastian terhadap nilai θ jauh lebih besar.

4. Selang kepercayaan (Confidence interval)

4.1 Konstruksi Selang Kepercayaan

Analisis ini dilakukan untuk mengukur tingkat ketidakpastian dari estimasi probabilitas keberhasilan *pull request*.

Besaran	Nilai
Tingkat kepercayaan	95%
Estimasi titik (θ)	0,8723
Batas bawah CI	0,8542
Batas atas CI	0,8904

Perhitungan ini dilakukan menggunakan rumus: $\theta \pm z(\sigma/\sqrt{n})$

Dengan:

- $\theta = 0,8723$
- $z = 1,96$
- $n = 1300$
- $\sigma = \sqrt{\theta(1-\theta)}$

Sehingga diperoleh selang kepercayaan 95% sebesar $CI_{95\%} = (0,8542 ; 0,8904)$

4.2 Interval Kredibel (Bayesian)

Pendekatan *Bayesian* juga digunakan untuk melengkapi analisis *frequentist* melalui distribusi posterior beta berdasarkan data yang diamati:

$$k = 1134$$

$$m = 166$$

Dan dengan prior beta(1,1), maka diperoleh parameter posteriornya:

$$\alpha = k + 1 = 1135$$

$$\beta = m + 1 = 167$$

Berdasarkan distribusi tersebut, interval kredibel 95% berada pada rentang yang sangat dekat dengan hasil *confidence interval* sebelumnya yang menunjukkan konsistensi antara pendekatan *frequentist* dan *Bayesian*.

4.3 Interpretasi

Dari hasil analisis yang menunjukkan bahwa estimasi probabilitas *pull requests* berhasil di-merge adalah sekitar 0,8723 atau 87,23% dan *Confidence Interval* 95% berada pada rentang 0,8542 hingga 0,8904 yang menunjukkan estimasi parameter memiliki tingkat ketidakpastian yang relatif rendah.

Rentang interval yang sempit disini disebabkan oleh jumlah observasi yang cukup besar dengan 1300 *pull request*. Hal ini menunjukkan estimasi yang diperoleh cukup stabil dan dapat mempresentasikan kondisi repository selama pengamatan. Dan dari pendekatan *Bayesian* dengan hasil nilai $\alpha = 1135$ dan $\beta = 167$. Hasil interval kredibel yang diperoleh konsisten dengan *confidence interval* sehingga ini memperkuat keyakinan bahwa probabilitas merge yang diestimasi mempresentasikan kondisi aktual repository.

Secara keseluruhan tingginya probabilitas merge menunjukkan bahwa proses review dan integrasi kontribusi pada repository telah berjalan dengan baik. Temuan ini juga mengindikasikan kalau *maintainer* mampu mengelola kontribusi secara efektif dan menjaga kualitas perubahan yang masuk ke proyek.

5. Uji hipotesis

5.1 Latar Belakang Pengujian

Dalam ekosistem pengembangan perangkat lunak modern berbasis *open-source*, manajemen *Pull Request* (PR) merupakan komponen vital yang menentukan kecepatan siklus rilis produk. Repository *microsoft/vscode* mengombinasikan kontribusi manual dari pengembang manusia (User) dan skrip otomatisasi (Bot) untuk mengelola alur kerja ini.

Pengujian inferensial ini bertujuan untuk membuktikan secara empiris apakah terdapat perbedaan yang signifikan dalam tingkat keberhasilan penggabungan kode (*Merge Rate*) antara kontributor User dan Bot. Evaluasi ini penting untuk mengukur efektivitas intervensi otomatisasi dalam arsitektur repository sistem informasi kelompok kami.

5.2 Metodologi Pengujian: Prosedur 6 Langkah Uji Hipotesis

Analisis pengujian menggunakan metode Uji Z Dua Sampel Independen (Two-Sample Z-Test). Pendekatan pengujian parametrik ini valid digunakan karena ukuran sampel dari kedua populasi sangat besar ($n_1 = 1124$ dan $n_2 = 176$), di mana keduanya > 30 berdasarkan Teorema Limit Pusat (*Central Limit Theorem*), dan standar deviasi dapat diestimasi dengan baik dari varians sampel.

Langkah 1: Formulasi Hipotesis (H_0 dan H_1)

- Hipotesis Nol (H_0): $p_1 = p_2$ atau $p_1 - p_2 = 0$
Hipotesis Nol ini menyatakan bahwa tidak ada perbedaan keberhasilan merge antara manusia dan bot.
- Hipotesis Alternatif (H_1): $p_1 \neq p_2$ atau $p_1 - p_2 \neq 0$
Hipotesis Alternatif atau (H_1) ini menyatakan bahwa terdapat perbedaan keberhasilan merge antara manusia dan bot.

Langkah 2: Menetapkan Tingkat Signifikansi (α)

Tingkat signifikansi yang digunakan dalam audit statistik ini ditetapkan sebesar $\alpha = 0.05$ (Tingkat Kepercayaan 95%). Ini adalah standar baku untuk meminimalkan risiko Kesalahan Tipe I (menolak H_0 yang sebenarnya benar).

Langkah 3: Menentukan Statistik Uji

Formula statistik uji yang digunakan untuk menghitung nilai signifikansi perbedaan dua rata-rata populasi independen adalah rumus Uji Z Dua Sampel:

$$Z = (\bar{X}_1 - \bar{X}_2) / \sqrt{((\sigma_1^2 / n_1) + (\sigma_2^2 / n_2))}$$

Keterangan: $\bar{X}$ adalah rata-rata sampel, σ adalah standar deviasi sampel, dan n adalah ukuran sampel kelompok.

Langkah 4: Menentukan Daerah Keputusan (Wilayah Kritis)

Berdasarkan tabel distribusi normal standar untuk uji dua arah (*two-tailed test*) dengan $\alpha = 0.05$, diperoleh nilai batas kritis $Z_{\text{tabel}} = \pm 1.96$.

- Kriteria Keputusan: Tolak H_0 jika $Z_{\text{hitung}} > 1.96$ atau $Z_{\text{hitung}} < -1.96$.

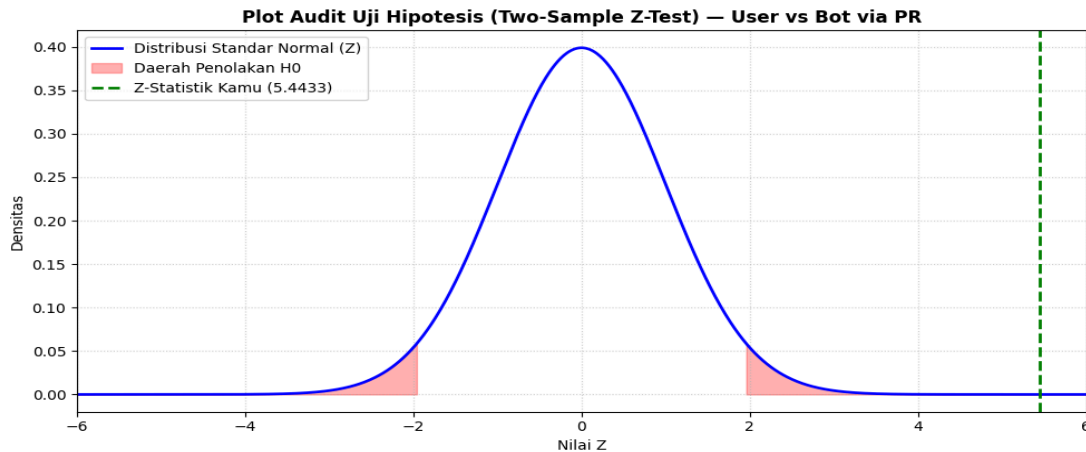
Langkah 5: Perhitungan Menggunakan Data Riil Komputasi

Melalui ekstraksi data bersih dari berkas `pull_requests_clean.csv`, diperoleh parameter deskriptif dua kelompok sebagai berikut:

- Populasi 1 (User / Manusia): Ukuran sampel (n_1) = 1124 | Rata-rata Merge Rate ($\bar{X}_1$) = 0.8986 (89.86%) | Standar Deviasi (σ_1) = 0.3020
- Populasi 2 (Bot / Otomatisasi): Ukuran sampel (n_2) = 176 | Rata-rata Merge Rate ($\bar{X}_2$) = 0.7045 (70.45%) | Standar Deviasi (σ_2) = 0.4575

Setelah parameter tersebut dihitung menggunakan algoritma statistik pada Jupyter Notebook, didapatkan hasil akhir:

- Nilai Z_{hitung} : 5.4433
- Nilai P-value: 0.0000 (5.23×10^{-8})



Gambar 5.2.1 Kurva Distribusi Normal Standar Uji Z Dua Sampel (User vs Bot)

Langkah 6: Keputusan Statistik

Berdasarkan hasil komputasi, nilai $Z_{\text{hitung}} = 5.4433$ berada jauh di dalam daerah penolakan ($5.4433 > 1.96$). Didukung oleh nilai $P\text{-value} = 0.0000$ yang nilainya jauh lebih kecil dari $\alpha = 0.05$, maka keputusan statistiknya adalah MENOLAK H_0 (Reject H_0).

5.3 Interpretasi Akhir dan Analisis Substantif

Berdasarkan hasil pengujian statistik yang dilakukan maka keputusan statistiknya ialah menolak H_0 karena $p\text{-value}$ -nya 0,0000. Hal inilah yang membuktikan bahwa persentase dari keberhasilan merger PR antara user dan bot itu berbeda jauh. Pada data yang ada, terlihat bahwa user sukses sampai 89,86% sedangkan bot hanya mencapai 70,45% saja. Maka dari itu performa manusia pada repositori VS Code lebih bagus dan akurat daripada bot.

6. Analisis komputasi

Analisis komputasi pada bagian ini bertindak sebagai jembatan integratif yang menerjemahkan temuan statistik teoretis dari layer-layer sebelumnya ke dalam solusi rekayasa perangkat lunak dan optimasi manajemen repositori. Eksperimen dilakukan menggunakan tiga pendekatan komputasi probabilistik yang saling melengkapi (Referensi: Tsun, 2020).

6.1 Simulasi Monte Carlo: Estimasi Durasi Terbukanya *Issue* (RQ3)

- Latar Belakang & Justifikasi:** Anggota A (EDA) menemukan bahwa variabel `days_open` (durasi *issue* terbuka) pada 590 sampel berdistribusi tidak normal dan condong ke kanan dengan tingkat kecondongan yang tinggi (**\$skewness = 1,6913\$**). Jika dihitung menggunakan pendekatan analitik berbasis asumsi normalitas, hasilnya akan bias. Simulasi Monte Carlo dipilih karena sifatnya yang *distribution-free* (tidak bergantung pada asumsi distribusi tertentu) dan melakukan penarikan sampel acak langsung dari distribusi empiris asli.

- **Formulasi Matematika:** Estimasi probabilitas ($\hat{p}$) dan Galat Baku (*Standard Error* / SE) dihitung melalui frekuensi kejadian dari total uji coba (N_{trials}):

$$\hat{p} = \frac{\text{Jumlah Uji Coba Sukses}}{N_{\text{trials}}}$$

$$SE = \sqrt{\frac{\hat{p}(1 - \hat{p})}{N_{\text{trials}}}}$$
Konvergensi simulasi dijamin oleh Hukum Bilangan Besar (*Law of Large Numbers*), di mana nilai $\hat{p}$ akan semakin mendekati probabilitas sejati seiring meningkatnya iterasi.
- **Hasil Eksperimen ($N = 50.000$):** Dari 50.000 kali simulasi penarikan acak untuk menguji *Research Question 3* (RQ3), jumlah kejadian *issue* yang tetap terbuka lebih lama dari 12 hari adalah 0. Diperoleh nilai estimasi probabilitas $\hat{p} = 0,0000$ dengan $SE = 0,000000$.
- **Interpretasi Relevansi:** Hasil simulasi menunjukkan konsistensi mutlak dengan baseline empiris awal. Hal ini membuktikan secara ilmiah bahwa mekanisme triase dan siklus resolusi kendala teknis pada repositori [microsoft/vscode](#) berjalan sangat responsif dan efisien, di mana seluruh *issue* berhasil diselesaikan dalam rentang waktu ≤ 12 hari.

6.2 Bloom Filter: Saringan Probabilistik untuk *PR Event Stream*

- **Latar Belakang & Motivasi:** Menjawab tantangan tingginya volume aliran data *Pull Request* (\$1.300\$ PR) serta adanya perbedaan performa *merge rate* yang menjomplang antara kontributor manusia dan *Bot* (sebagaimana dibuktikan oleh Anggota D melalui uji hipotesis $Z = 5,4433$), diimplementasikan struktur data probabilistik *Bloom Filter*. Struktur data ini digunakan untuk menyaring duplikasi *event stream* secara *real-time* pada pipeline CI/CD dengan efisiensi memori yang sangat tinggi dibandingkan menggunakan struktur data *Set* konvensional.
- **Formulasi Teoretis False Positive Rate (FPR):** Struktur data ini menjamin **nol False Negative**. Namun, terdapat risiko *False Positive* (item baru salah dianggap sudah ada) yang dikalkulasi secara teoretis melalui rumus:

$$FPR \approx \left(1 - e^{-kn/m}\right)^k$$
Di mana n adalah jumlah item, m adalah ukuran *bit array*, dan k adalah jumlah fungsi *hash*.
- **Parameter & Hasil Pengujian:** * Jumlah elemen (n) = \$1.300\$ PR
 - Jumlah fungsi *hash* (k) = 3 fungsi (berbasis algoritma kriptografi [hashlib](#))
 - Ukuran *bit array* (m) = \$13.000\$ bit (menggunakan aturan praktis optimalisasi memori $10 \times n$)
 - **FPR Teoretis: \$0,000862\$ (atau \$0,0862\%)**
- **Uji Keanggotaan (*Membership Test*):** Pengujian aktual terhadap 5 sampel PR asli menghasilkan status 100% **FOUND** (lolos jaminan akurasi), dan 5 sampel ID PR palsu menghasilkan status 100% **Correctly rejected**.
- **Interpretasi Relevansi:** Dengan nilai tingkat kesalahan (*false positive*) teoretis di bawah \$0,1\%\$, filter ini sangat aman dan efisien untuk diterapkan di gerbang awal pemrosesan data *streaming tracking* repositori tanpa membebani kapasitas memori server.

6.3 MCMC Knapsack: Prioritisasi Portofolio Intervensi Repositori

- **Latar Belakang & Pemodelan Batasan:** Tim telah merumuskan 8 usulan proyek intervensi teknis berdasarkan temuan dari seluruh lapisan analitik sebelumnya. Namun, total kebutuhan waktu pengerjaan proyek tersebut (**67 hari-orang**) melampaui batas kapasitas kerja tim dalam satu siklus sprint (**35 hari-orang**). Algoritma *Markov Chain Monte Carlo* (MCMC) dijalankan sebanyak **\$100.000\$ iterasi** untuk mencari kombinasi biner (0/1) portofolio terbaik yang memaksimalkan nilai dampak tanpa melanggar batasan kapasitas kerja (*constraint*).
- **Hasil Optimasi Stokastik MCMC:** Dari ruang eksplorasi kombinasi yang luas, algoritma berhasil mengonvergensi tiga proyek intervensi dengan nilai efisiensi tertinggi:
 - *Bot PR quality gate (CI lint+test wajib)* — Bobot: 15 hari-orang | Nilai Dampak: 28

- *Filter/validasi PR Bot sebelum masuk review* — Bobot: 8 hari-orang | Nilai Dampak: 20
 - *Review latency monitoring dashboard* — Bobot: 10 hari-orang | Nilai Dampak: 18
- **Metrik Portofolio Optimal:**
 - **Total Nilai Dampak Akumulatif: 66**
 - **Total Alokasi Bobot Kerja: 33 hari-orang** (dari kapasitas maksimal 35)
 - **Sisa Kapasitas Cadangan (*Slack Capacity*): 2 hari-orang**
 - **Status Kelayakan Batasan: PASSED** (Lolos Uji Batas)
- **Interpretasi Relevansi:** Hasil optimasi stokastik ini sangat logis dan sejalan dengan temuan Anggota D. Mengingat *Bot merge rate* hanya berada di angka 70,45%, algoritma MCMC secara cerdas menempatkan proyek automasi pengujian *Bot* (skor dampak: 28 dan 20) sebagai prioritas utama yang harus dieksekusi tim pada sprint berikutnya guna mendongkrak kualitas kode repositori secara instan.

7. Rekomendasi

- 7.1 Temuan menunjukkan bahwa PR dari bot memiliki tingkat merge yang lebih rendah, yaitu 70,5% dibandingkan 89,9% milik kontributor user selisih sekitar 19 poin persen. Perbedaan ini bisa jadi mengindikasikan bahwa sebagian PR bot membutuhkan pengecekan lebih teliti sebelum di-merge. Namun, data ini tidak dapat memastikan penyebabnya. Karena itu, kami menyarankan maintainer mengevaluasi ulang jenis PR yang dibuat bot untuk memastikan apakah perbedaan ini wajar atau menandakan masalah yang perlu ditangani.
- 7.2 Hasil simulasi Monte Carlo menunjukkan bahwa sekitar 20% issue masih terbuka lebih dari 3 hari. Hal ini mengindikasikan bahwa ada cukup banyak issue yang belum tertangani dalam waktu yang relatif lama. Karena itu, kami menyarankan maintainer menerapkan sistem pelabelan (labeling) agar issue yang penting dapat lebih cepat diidentifikasi dan ditangani.

8. Penemuan utama

1. Tingkat keberhasilan merge Pull Request sangat tinggi
 - Dari 1.300 Pull Request yang dianalisis, sekitar 87,23% berhasil di-merge.
 - Confidence Interval 95% berada pada rentang 85,42%–89,04%, menunjukkan estimasi yang stabil dan tingkat ketidakpastian yang rendah.
2. Hasil estimasi Bayesian memperkuat temuan MLE
 - Distribusi posterior Beta menghasilkan nilai yang sangat dekat dengan estimasi MLE.
 - Konsistensi antara pendekatan Frequentist dan Bayesian meningkatkan keyakinan bahwa probabilitas merge yang diperoleh merepresentasikan kondisi repositori secara akurat.
3. Terdapat perbedaan signifikan antara performa User dan Bot
 - Merge rate User mencapai 89,86%, sedangkan Bot hanya 70,45%.
 - Uji Z menghasilkan $Z = 5,4433$ dan $p\text{-value} < 0,05$, sehingga perbedaan tersebut terbukti signifikan secara statistik.
4. Kontribusi manusia lebih efektif dibandingkan bot

- Pull Request yang diajukan oleh manusia memiliki peluang diterima lebih tinggi dibandingkan Pull Request yang dibuat oleh bot.
 - Temuan ini mengindikasikan perlunya evaluasi terhadap kualitas dan proses otomatisasi bot.
5. Pengelolaan issue berjalan sangat responsif
 - Simulasi Monte Carlo sebanyak 50.000 iterasi menunjukkan probabilitas issue tetap terbuka lebih dari 12 hari sebesar 0%.
 - Seluruh issue dalam sampel berhasil diselesaikan dalam waktu ≤ 12 hari.
 6. Bloom Filter terbukti efisien untuk penyaringan data
 - False Positive Rate (FPR) teoritis hanya 0,0862%.
 - Pengujian menunjukkan seluruh data valid terdeteksi dengan benar dan data palsu berhasil ditolak.
 - Solusi ini layak digunakan untuk pemrosesan event Pull Request secara real-time dengan penggunaan memori yang rendah.
 7. Prioritas perbaikan repositori berfokus pada kualitas bot
 - Optimasi MCMC Knapsack mengidentifikasi tiga intervensi paling berdampak:
 - Bot PR Quality Gate (CI lint & test wajib)
 - Validasi Pull Request bot sebelum review
 - Dashboard monitoring waktu review
 - Ketiga proyek tersebut memberikan nilai dampak tertinggi dengan tetap memenuhi batas kapasitas kerja tim.
 8. Kesimpulan utama
 - Repositori VS Code memiliki proses review dan integrasi kontribusi yang sangat baik, ditunjukkan oleh merge rate yang tinggi dan penyelesaian issue yang cepat.
 - Area yang paling membutuhkan perhatian adalah peningkatan kualitas Pull Request yang dihasilkan oleh bot agar performanya dapat mendekati kontribusi manusia.